

District IQ

Complete Review Guide

Chennai Intelligent District Governance Platform
FarmWise · FarmwiseAI Campus Product Challenge, Task 6

Everything end to end, in simple words: what we built, how every dashboard feature works, where each piece of data comes from, every algorithm (how it works, why we chose it, why it is better), the full tech stack, and the questions reviewers are likely to ask, with answers.

Prepared 2 October 2026 · Matches repository [ApshanaSP/farmwisenew](#) at commit [dda6e99](#)

Contents

How to use this guide	3
Part 0. What we did to set it up on this PC	4
Part 1. The big picture	5
1.1 The problem	5
1.2 Our goal (five things)	5
1.3 The architecture: seven steps from source to decision	5
1.4 The journey of one piece of information	5
Part 2. Data collection: every source	7
Part 3. The news pipeline	9
Part 4. district_intel: the intelligence layer	10
Part 5. The web application and every dashboard feature	11
5.1 Login and roles	11
5.2 Citizen portal	11
5.3 Collector console (/collector): the main showpiece	11
5.4 Department officer consoles (/officer): 25 departments	13
5.5 Ask District IQ: the AI assistant	14
Part 6. Every algorithm, explained simply	15
Part 7. Storage: where the data lives	23
Part 8. Security and privacy	24
Part 9. Tech stack: what, how we used it, why, why better	25
9.1 Data collection	25
9.2 Intelligence layer	25
9.3 Web application	26
9.4 AI layer	26
9.5 Operations	26
Part 10. Key design decisions (why we built it this way)	27
Part 11. Testing, results and honest limits	28
Part 12. Likely reviewer questions and answers	29
Glossary: technical words in plain English	31
One-page cheat sheet: numbers to remember	32

How to use this guide

Read it once from start to end, then revise with the green and yellow boxes:

- **Green boxes** (Say this in the review) are short lines you can say out loud.
- **Yellow boxes** (If they ask...) are likely reviewer questions with short answers.
- **Blue cards** in Part 6 explain each algorithm the same way: what it does, an everyday comparison, the steps, the real settings in our code, why we chose it, and why it beats the obvious alternative.
- The **last pages** are a glossary of technical words and a one-page cheat sheet of numbers to remember.

The whole project in 60 seconds

Say this in the review

A District Collector gets information from many places that do not talk to each other: citizen complaints, police, PWD, hospitals, weather warnings, pollution data and the news. The same flooded street can show up as four separate records in four departments.

District IQ collects all of these automatically, cleans them, puts every record on the real map of Chennai's 200 wards, removes duplicates, and links records from different departments that describe the same real-world incident. It then ranks incidents by priority and **says why** in plain words, finds unusual spikes and hotspots, and shows everything on a Collector dashboard and 25 department dashboards. Officers do the work, send proof with a photo, and the Collector verifies it. An AI assistant answers questions in English, Tamil or Tanglish.

Our core rule: **every number comes from the database and statistics, never from an AI model**. The AI only understands questions and writes sentences, and a verifier checks every number it says.

The core principle to remember

Numbers from SQL and statistics. AI only for language.

Language models can say confident but wrong numbers. In government, one wrong number can mean a wrong decision. So in District IQ, counts, rankings and trends are always computed by SQL queries and statistical methods. The AI is used only to understand the question and to write the answer as a sentence, and a number verifier rejects any answer containing a number that is not in the data. Without any AI key the whole system still works.

Part 0. What we did to set it up on this PC

This is the setup journey on your computer (D:\farm), in order. Reviewers sometimes ask "how do you run it?" and this part answers that, including the problems we hit and how we solved them.

#	What we did	Why / problem solved
1	Downloaded the repository ApshanaSP/farmwisenew from GitHub as a ZIP (no Git on the PC) and extracted it into D:\farm.	First attempt with PowerShell's downloader was very slow; switched to curl.exe + tar, which took seconds.
2	Downloaded the 16 large files separately (AI model 278 MB, news dumps, district_intel.db 160 MB, about 950 MB in total).	GitHub ZIPs do not include Git LFS (Large File Storage) files; they contain small 'pointer' text files instead. We fetched the real files from GitHub's LFS media server.
3	Installed the web app's packages: npm install in chennai-grievance-portal-main.	Next.js, React and all other libraries the website needs.
4	Created the .env settings file from .env.example with a random JWT secret and Aadhaar encryption key.	.env holds secrets and is never committed to GitHub.
5	Installed MySQL 8.4 as a portable copy in D:\farm\mysql, root password 'arjun', then registered it as the Windows service MySQL84 that starts automatically.	The app stores users, complaints and the dashboard data in MySQL.
6	Created the database and ran the setup scripts: db:setup (11 migrations, 250 areas, 4,883 localities, 53,800 streets, 200 wards), setup:ops, setup:officer, seed:contacts (48 GCC officials), seed:officers (25 department logins), seed-users (sample Collector/citizen logins).	Builds every table the portal needs.
7	Created a Python virtual environment for district_intel and loaded the intelligence store (district_intel.db) into MySQL: 31 tables, about 300,000 rows.	The Collector dashboard reads this store.
8	Fixed: Windows Smart App Control blocked a scikit-learn file. Installed scikit-learn 1.8.0, which Windows allows.	Needed for the pipeline's machine learning steps.
9	Ran the data pipeline: refresh (8 collectors) + build + MySQL export.	Fresh news, weather, air quality, reservoirs, hospital, PWD and police data.
10	Applied the friend's commit dda6e99 only (53 files + one removed file), verified every file against GitHub's checksums, backed up the old versions in D:\farm_backup_before_dda6e99, and reloaded MySQL.	Rolling 24-hour Daily window, Collector news tab, hourly collection, new login cookies.

How to run it now

Task	Command / action
Start the website	cd D:\farm\chennai-grievance-portal-main then npm.cmd run dev then open http://localhost:3000
Log in as Collector	collector@chennai.gov.in / Passw0rd! (officer logins are in officer-accounts.local.csv)
Stop the website	Ctrl + C in the terminal
MySQL	Runs automatically as the Windows service MySQL84
Refresh data + rebuild	cd D:\farm\district_intel ; .venv\Scripts\python.exe run_pipeline.py refresh --mysql
Rebuild only	python run_pipeline.py build (then python run_pipeline.py mysql)
Tests	python -m pytest -q tests (Python) and npx vitest run (website)

Part 1. The big picture

1.1 The problem

A District Collector receives information from many disconnected channels: the citizen grievance portal, police station reports, PWD flood and works registers, hospital systems, IMD weather warnings, the pollution board and the daily news. Each has its own format and its own IDs, and none talks to the others. **The same waterlogged street may appear as a grievance, a police call, a PWD incident and a news story: four records, one problem.**

1.2 Our goal (five things)

- **Collect** all sources automatically.
- Turn them into **one trustworthy picture of real incidents**.
- Tell the Collector **what needs attention today and why**.
- **Route work** to departments and track it to **verified completion**.
- **Answer questions** in plain language, with every figure traceable to its source.

1.3 The architecture: seven steps from source to decision

Each layer has one job, and a lower layer never depends on an upper one. So collectors keep running even when the dashboard changes, and the dashboard can be rebuilt without fetching the data again.

1. Sources	Google News + publisher RSS, IMD weather, CPCB air quality, CFM-DSS flood monitor, AGMARKNET market prices, GCC officials; synthetic police, PWD, hospital and grievance records
2. Collectors (Python, Node.js)	One program per source. Each writes CSV/JSON files with provenance (where each row came from). Rolling 90 to 180 day windows.
3. district_intel pipeline (Python)	classify, geo-locate, scenario overlay, remove duplicates, link across sources, build incidents, analytics, agents
4. Curated store	SQLite file district_intel.db (32 tables + 6 views), copied into MySQL (or AWS S3/DynamoDB in the cloud version)
5. Next.js web application	Citizen portal, Collector console, 25 department consoles, REST APIs, PDF/CSV exports
6. Ask District IQ	Guard, router, tools or safe SQL, facts, composer, number verifier, chart checker
7. People decide	Officers approve and complete work; the Collector verifies or returns it; every action goes into the audit log

1.4 The journey of one piece of information

Example: heavy rain causes waterlogging in a ward. This is what happens, step by step.

#	Step	What happens	Where in the code
1	Schedule fires	Windows Task Scheduler runs the pipeline hourly and at sign-in (since commit dda6e99; before it was daily at 6 AM).	district_intel/scripts/run_intel.bat

#	Step	What happens	Where in the code
2	Collectors fetch	IMD rainfall and warnings, CFM-DSS river and lake levels, CPCB AQI, news RSS. Each source is tried 3 times; failures are logged.	*_collector.py, chennai_news_pipeline
3	Shared rain calendar	Rain days are passed to the synthetic generators so every source agrees on the weather.	dintel/sync.py
4	Generators refresh	Grievance, police, PWD and hospital records roll forward (180-day window).	generator scripts
5	News processed	Cleaned, language detected, Chennai-only filter, tagged with one of 15 departments.	chennai_news_pipeline/src
6	Load + normalise	8 loaders map every source into one 70-column 'events' table. Mobile numbers are replaced by a keyed hash (privacy).	dintel/loaders/*
7	Classify	Category from lookup tables for structured data; machine-learning classifier for free text.	dintel/classify.py
8	Geo-locate	Put each record in its ward (200 GCC wards), zone and taluk.	dintel/geo.py, geolocate.py
9	Remove duplicates	Same-source repeats grouped with union-find.	dintel/dedup.py
10	Link across sources	Grievance + police + PWD + news about the same event become one incident.	dintel/linking.py
11	Incidents	Lead record, status, deadline, priority score with reasons.	dintel/incidents.py, severity.py
12	Analytics	Spikes (Poisson), hotspots (DBSCAN, Getis-Ord Gi*), sensor trends (EWMA, z-score), KPIs.	dintel/analytics.py
13	Agents	Data Steward, Action Planner, Gap Finder, Linker, Watchdog, Briefing writer.	dintel/agents/*
14	Store + export	SQLite + CSV + JSON, then MySQL with an atomic swap.	dintel/store.py, mysql_export.py
15	Dashboard shows it	On the map, in By severity and in My Tasks, with plain reasons.	src/lib/collector/intel.ts
16	Department acts	Officer approves, works, completes with remarks and a site photo.	src/app/officer, src/lib/officer
17	Collector verifies	Verify or Send back with a note; written to the audit log.	/api/collector/decisions
18	Ask and report	Ask District IQ answers questions; PDF / CSV exports.	src/lib/assistant, reportPdf.ts

Say this in the review

A single rain event travels from the IMD website and a citizen's complaint, through our pipeline where it becomes one incident with a priority and reasons, onto the Collector's map, to the right officer, and back to the Collector for verification, and every step is logged.

Part 2. Data collection: every source

A firm project rule: **collectors are never edited for the dashboard's needs**. Each collector owns exactly one source, never bypasses CAPTCHAs or logins, and labels fallbacks clearly. Any data fix is done later, in `district_intel`. This keeps each collector simple and testable on its own.

Source	Real or synthetic	What it gives	How we fetch it	Limits (be honest)
News (chennai_news_pipeline)	Real	Chennai news in English and Tamil, about 20,600 articles	Google News RSS + publisher RSS (feedparser); article text with <code>trafilatura</code>	About 98% are headline snippets; full text only for some
IMD weather	Real	Rainfall, temperature, humidity, 7-day forecast, district warnings with colour	IMD city-weather JSON API, GeoServer WFS warnings, RMC Chennai pages	4 stations; the AWS/ARG portal needs login, which we do not bypass
CPCB air quality	Real	AQI, category, main pollutant per station	<code>data.gov.in</code> API, else Playwright reads the public CCR map	History is CAPTCHA-gated, so history builds up from our own runs
CFM-DSS (TN Water Resources flood monitor)	Real	River/canal gauge levels with warning stages, lake storage, inflow, alerts	<code>chennaifloodmonitor.tn.gov.in</code> JSON endpoints	No history endpoint; history built from our runs
AGMARKNET	Real	Vegetable and food prices at Chennai Uzhavar Sandhais	AGMARKNET open APIs	Koyambedu does not report
GCC officials	Real	48 officials: names, roles, phones	<code>chennaicorporation.gov.in</code> who-is-who page	Seeded once into <code>official_contacts</code>
Citizen grievances	Synthetic + real	Complaints with category, ward, pin, status history (about 8,100)	Portal generator script + real submissions	Uses the shared rain calendar
Police	Synthetic	16 taluks, 60 stations, about 12,300 incident reports	Node.js + TypeScript generator	Duplicate ground truth kept separately
PWD	Synthetic	Flood incidents, works register, field records, lakes	Python generator	180-day window
Government hospitals	Synthetic	Bed occupancy, OPD, emergencies, disease cases, medicine stock, ambulances	Python generator	180-day window

Why some data is synthetic

Police, PWD, hospital and most grievance data are not published openly, so we generate realistic records. Every synthetic row is flagged (`is_synthetic = 1`). To plug in a real department system later, we only write one new loader; nothing else changes.

How collection is scheduled

- **On the PC:** Windows Task Scheduler runs `district_intel/scripts/run_intel.bat`. Since the latest commit, every source is collected **hourly** (`config.yaml` `every_minutes: 60`), and also at sign-in, so a PC that was off overnight catches up in the morning.
- **Retries:** each source is tried up to 3 times, 60 seconds apart, because a site timing out usually works a minute later.
- **Cloud version:** GitHub Actions calls AWS API Gateway on a timer (hourly for IMD/CFM/PWD/hospital, every 3 hours for news).

If they ask...

Q: Is the data real?

A: News, IMD, CPCB, the flood monitor, AGMARKNET prices and GCC officials are real. Police, PWD, hospital and most grievances are synthetic because those departments do not publish open data. Every synthetic row is flagged, and connecting a real source means writing one loader.

Q: What happens if a source fails?

A: It is retried 3 times. Failures are logged, and the feed chip on the console shows how old the data is, so the Collector sees stale data instead of trusting it blindly.

Q: Did you bypass any CAPTCHA or login?

A: No. That is a project rule. Where history is CAPTCHA-gated (CPCB), we build history from our own runs.

Part 3. The news pipeline

News is the only source that reports what the departments have not recorded. The news pipeline deliberately stays simple: it fetches, cleans, filters for Chennai and tags a department. It does **not** deduplicate or cluster; that is district_intel's job, so each step can be tested on its own.

Step	What we do	Why
Fetch	Google News RSS queries + publisher RSS feeds. 90-day backfill on the first run, then the last 48 hours.	No API keys, stable, covers English and Tamil outlets.
Extract	trafilatura downloads the article page when allowed and pulls out just the article text.	Better text than the short RSS summary.
Clean	Strip HTML, normalise Unicode, detect language (langdetect), convert dates to Indian time (Asia/Kolkata).	Tamil text and IST times shown correctly.
Relevance (A1)	A gazetteer (list) of Chennai places in English and Tamil, plus exclusion rules.	Removes national news and other districts.
Classify (A2)	Keyword model gives one of 15 departments and a complaint flag.	So news can be routed to the right department.
Store	Rolling 90-day master_news.csv plus dated raw and processed files.	Can reprocess without fetching again (--reprocess).

Tests: 88 pytest tests. HTTPS behind antivirus TLS inspection works via truststore (uses the Windows certificate store, so we never switch off certificate checks).

What changed in the latest commit (dda6e99)

- The Briefing's **News tab** now lists only stories that matter to the Collector, from real news outlets (established, regional or official), not social posts, aggregators or daily round-ups like "Chennai Latest News Today".
- **One row per story**, with every outlet's article merged. Each row says whether the story is also a citizen grievance, an incident on record, or news only.
- The incident pop-up now tags news coverage ("In the news, 3 outlets") and the number of citizen grievances, and "How it unfolded" lists the grievances and news reports together in time order.

Part 4. district_intel: the intelligence layer

This is the heart of the project. It reads every collector's output and builds one curated, traceable, dashboard-ready store. A full build takes about 3.5 minutes on a laptop CPU. Every threshold lives in config.yaml, so it can be tuned without changing code.

Stage	What it does	Algorithm (Part 6)
Load	8 loaders put every source into one 70-column 'events' table. Each row keeps its source file (provenance). Mobile numbers become a keyed hash.	-
Category	Lookup tables (crosswalks) for structured sources; machine-learning classifier for free text and 'Other' complaints.	A3
News stories	One document per URL; Google copies merged into the publisher's copy; same-event articles grouped into stories.	A4, A5
Location	Each record into its ward, zone and taluk using the real 200 GCC ward boundaries.	A6
Scenario overlay	Makes the separately generated synthetic sources agree on the same rain days; every planted row is flagged.	A7
Duplicates	Same-source repeats grouped together.	A8
Cross-source linking	Records from different departments about the same event become one incident.	A9
Incidents	Lead record, status, deadline, severity, priority with reasons, attention flags.	A10
Analytics	Spikes, hotspots, hot wards, sensor trends, KPIs.	A11 to A14
Agents	Briefing, Data Steward, Action Planner, Gap Finder, Linker, Watchdog.	A15, A16
Outputs	district_intel.db, curated CSVs (UTF-8 with BOM so Tamil opens in Excel), dashboard JSON, reports.	A20

The main tables (what the dashboard reads)

Table	One row per
events	source record: a complaint, police report, PWD incident, hospital alert, IMD warning, or news incident
incidents, incident_members, incident_timeline	real-world incident, the records merged into it, and its combined timeline
actions	task (real PWD tasks + Action Planner drafts)
documents	news article, PWD announcement or flood bulletin, with its story cluster
observations, observation_signals	sensor/metric reading (hospital beds, lakes, IMD, CPCB, gauges) and its latest signal
kpis, daily_counts, anomalies, hotspots	headline number, daily count, detected spike, detected cluster
alerts, briefings, gaps, review_queue	agent outputs (drafts; people decide)
source_health, data_quality	Data Steward checks
ref_* and world_calendar	reference lists (wards, taluks, departments, categories, facilities) and the shared day calendar

Part 5. The web application and every dashboard feature

The website is one Next.js 14 application (chennai-grievance-portal-main) with three kinds of users. When you open `http://localhost:3000, src/app/page.tsx` checks who is logged in and sends them to their home: a citizen to `/citizen`, a department officer to `/officer`, the Collector to `/collector`.

5.1 Login and roles

- Passwords are hashed with **bcrypt** (never stored as plain text).
- After login the server gives the browser a **JWT** (a signed token) inside an **httpOnly cookie**, which JavaScript on the page cannot read, so it is protected from cross-site scripting.
- **Middleware** (`src/middleware.ts`) runs before every page and blocks anyone whose role does not match the page.
- **New in dda6e99**: each portal (citizen, officer, Collector) has its own session cookie (`src/lib/session-cookies.ts`), so you can be logged in to the Collector and an officer console in the same browser without one logging out the other.
- Login rate limits: 20 attempts per IP and 8 per email per 15 minutes.

5.2 Citizen portal

Feature	How it works	Why
Register / login	Email OTP, password reset, profile; inputs checked with zod on both browser and server.	Only verified users can file complaints.
File a complaint	Verified GCC complaint types; area, locality, street drop-downs; a Leaflet map pin; photo upload. The ward is found from the pin using the real ward boundary polygons.	Accurate location means the right department and ward.
'Others' category	If the citizen picks 'Others', an AI classifier suggests the category, with a keyword classifier as the fallback when no AI key is set.	Complaints still get routed even when the citizen does not know the category.
Complaint code	Each complaint gets a unique code to track it.	Easy follow-up.
Track complaints	Status and full history of each complaint.	Transparency for the citizen.

Code: `src/app/citizen/file-complaint/page.tsx` (the form) and `src/app/api/complaints/route.ts` (validates, finds the ward, classifies, creates the code, saves).

5.3 Collector console (`/collector`): the main showpiece

A **fixed screen with no scrolling**: everything the Collector needs fits on one screen, in four pages. Every number on the console comes from a SQL query in **`src/lib/collector/intel.ts`**; the screens are React components in `src/components/collector/app`.

Top bar

Element	What it does / how
Search	Search incidents, zones and departments.
Feed status chip	Shows how fresh each data feed is, so the Collector knows whether to trust it.
Bell	Notifications (alerts from the pipeline).
Export	PDF briefing report and CSV action list.
Customize (sliders icon)	Turn pages, panels and headline cards on or off; Apply or Reset to default.
Ask District IQ	Bottom-right launcher or Ctrl+K opens the AI assistant.

Filters

Filter	How it works	Why designed this way
Period: Daily / Weekly / Monthly / Quarterly	Daily = the last 24 hours ending now (since commit dda6e99; opened at 5:30 PM it covers yesterday 5:30 PM to now). Weekly, Monthly, Quarterly = last 7, 30, 90 days. Each card compares with the previous period of the same length.	A rolling 24 hours always has a full day of data; 'today from midnight' looked empty early in the morning.
Zone (15 zones)	Filters every panel; the map zooms to the zone.	Matches how GCC is organised.
Taluk (17 taluks)	Filters every panel by revenue taluk.	Matches the revenue administration.
Department	Filters every panel on the same dashboard; the drop-down shows how many open items each department has.	We deliberately did not make separate pages per department.
Clear	Resets all filters.	Filters combine; clear before trying a new one.

Tip for the demo

On **Daily**, narrow filters often show 0 because only the last 24 hours count. Switch to **Weekly** or **Monthly** to show filters clearly changing the numbers. Only the zone filter zooms the map; taluk and department change the numbers but not the map view.

Page 1: Overview

Panel	What it shows	How it is computed
Headline cards	Severe events, Open complaints, Ongoing incidents, Resolved , each with the change versus the previous period and a small trend line.	SQL over the incidents table in the period: Severe = incidents with severity Severe; Open complaints = citizen complaints on open incidents; Ongoing = open incidents; Resolved = closed with status Resolved.
Satellite map	All 200 ward and 15 zone outlines on Esri satellite imagery; incident pins coloured by type; violet pins = items from added sources. Zoom and pan; Zones / Taluks toggle.	Leaflet + Esri imagery; ward polygons from GCC boundary data; pins from incident latitude/longitude.
District Snapshot	Strip of headline numbers for the district or the selected zone.	SQL aggregates.
By severity	Open incidents grouped Severe, High, Medium, Low (up to 12 per group), highest priority first.	Severity and priority from the pipeline (A10).
My Tasks	The Collector's personal to-do list. Officers' completion reports always come first. Then only closures worth the Collector's time.	Included if: Severe or High severity, 3 or more citizens complained, several departments involved, in the news, or Medium but over twice its deadline. Each task shows its reason.
Today's Briefing	Short written summary, plus the News tab (real outlets only, one row per story) and added sources.	Briefing agent (A15) and the news rules from dda6e99.

Incident pop-up (click any incident)

- Plain-language summary: what happened, key facts, **why it needs attention** (the priority reasons).
- Who is responsible, with the real official's name from the GCC website.
- **How it unfolded**: every record merged into this incident in time order (grievance, police report, news). This is cross-source linking you can see.
- News coverage counted by **distinct publisher**, so ten copies of one article do not look like ten newspapers.
- The department's completion report with remarks and photos, and the **Verify / Send back** buttons.

Page 2: Briefing

- The readable briefing (daily, weekly, monthly or quarterly).
- **Developing stories:** news threads with two or more articles about the same event (story clustering, A5).
- **Follow-ups.**
- **Only in news:** incidents reported in the news that no department has recorded. This is the Collector's blind spot. The Gap Finder (A16) finds them and routes them to the right department's officer.

Page 3: Trends

- District-wide category rows by week, using full Monday-to-Sunday weeks so comparisons are fair.
- **Unusual spikes** from the Poisson model (A11). **Recurring hotspots** from DBSCAN (A12). **Unresolved by taluk.**
- Clicking a row opens a detail view explaining the pattern: observed versus expected, and where it is happening.

Page 4: Environment and markets

- Rainfall, air quality (AQI) and reservoir cards that follow the selected zone. If a zone has no station, the nearest station is used **and named with its distance**, so the reading is honest about where it comes from.
- Lake levels from the flood monitor; sensor signals use EWMA and z-scores (A14).
- Vegetable prices at each Chennai Uzhavar Sandhai, with the cheapest and dearest market (AGMARKNET).

Exports, decisions and audit

- **PDF briefing** generated in the browser with jsPDF + autotable (top 10 items), so there is no load on the server.
- **CSV action list:** not a raw dump, but the things that need the Collector plus closures to check.
- **Verify / Send back** decisions are written to collector_decisions, and every action to **audit_log**, in the separate district_intel_ops database, which the pipeline never overwrites.
- **Added sources:** the Collector can add a source (for example an RSS feed, or a scanned newspaper page read by tesseract.js OCR in the browser, English + Tamil); its items appear as violet pins.

Say this in the review

The Collector console is one fixed screen. The map shows where, By severity shows how bad, My Tasks shows what needs me and why, and the briefing tells the story. Every number comes from a SQL query, and every incident explains its own priority.

5.4 Department officer consoles (/officer): 25 departments

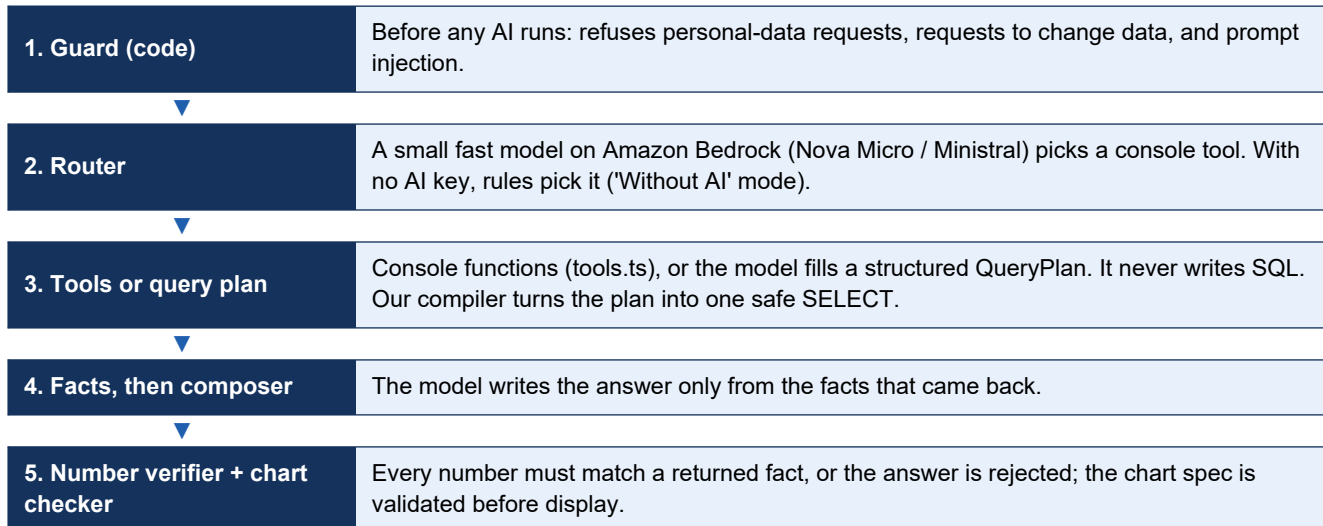
- One account per department (officer.<code>@chennai.gov.in) with its own password.
- The department is read from the database on **every request**, never from the URL, so an officer cannot see another department's data by changing the address. Other departments' records return 'not found'.
- **Page 1, Overview:** the Collector's overview filtered to the department: Severe events, Open incidents, Waiting for approval, Resolved, map, snapshot, My Work, briefing.
- **Page 2, Work and insights:** grievance board (New, In action, Sent to Collector, Verified), 'What needs you now', complaint-type changes, and department-specific insight cards: police see crime and response time, hospitals see beds and medicine stock, PWD sees lakes and works, TNPCB sees air quality, Revenue sees market prices, roads/electrical/parks/TANGEDCO see rainfall.
- **Task pipeline:** New, Approve, In action, Complete and send (remarks + site photo), Sent to Collector, Verified, or Returned with the Collector's note.
- Photos are stored outside the public folder and served only to the Collector and the sending department.

Say this in the review

This closes the loop: citizen, pipeline, officer, Collector, audit. Work is tracked until the Collector has verified it with proof.

5.5 Ask District IQ: the AI assistant

A pop-up on both consoles (bottom-right or Ctrl+K). It answers in English, Tamil or Tanglish, with charts (ECharts), maps and tables. Follow-ups edit the previous answer ("only Zone 13", "make it a pie", "show on map"). Pinned answers re-run on fresh data. "Insights for today" gives a quick summary. Limits: 20 questions per minute, 300 per day per user, CSV export up to 1,000 rows.



For officers, answers are **locked to their department on the server** (lockDept). A ranking question becomes their own department's figures. No prompt can get around it, because the lock is in the code, not the prompt. Details: algorithms A17 to A19.

On your PC no AI keys are set, so the assistant runs in 'Without AI' rule mode; maps use OpenStreetMap; email sending is off.

Part 6. Every algorithm, explained simply

Twenty methods (A1 to A20). For each: what it does, an everyday comparison, the steps, the real settings from our code, why we chose it, why it beats the obvious alternative, and the measured result where we have one. Settings were checked against the current code (config.yaml, dintel/*.py, src/lib/assistant/*).

Group 1: Understanding news and text

A1. Chennai relevance filter (gazetteer matching)

Used in	chennai_news_pipeline/src/relevance_filter.py
Purpose	Keep only news about Chennai district out of thousands of RSS items.
Simple picture	Like a postman with a list of every Chennai street and landmark: if the letter mentions one, it is ours; if it mentions another district, it is not.
How it works	1) Detect the language (langdetect) so Tamil and English rules apply correctly. 2) Match the headline and text against a gazetteer: Chennai places, localities and landmarks in English and Tamil script, with spelling variants. 3) Exclusion rules drop other districts and national stories.
Why we chose it	News feeds mix national and other-district news; the dashboard must show Chennai only.
Why it is better	Versus a trained location model: no labelled training data existed; a list is fully explainable, handles Tamil script directly, and is fixed by just adding a place name (then --reprocess, without fetching again).

A2. Department tagging of news (keyword model)

Used in	chennai_news_pipeline/src/classification.py
Purpose	Tag each article with one of 15 departments, plus a complaint flag, so it can be routed.
Simple picture	Like sorting mail by keywords on the envelope: 'garbage' goes to Solid Waste, 'streetlight' to Electrical.
How it works	Each department has weighted keyword lists (English + Tamil). The article gets a score per department; the highest score wins. A complaint flag is set when civic-failure words appear.
Why we chose it	Needed from day one, before any labelled data existed; must also run inside a small cloud function.
Why it is better	Versus asking an AI model per article: free, instant for 20,000+ articles, gives the same answer every time and is testable (88 tests); no API cost or quota.

A3. Category classifier: character n-gram TF-IDF + Logistic Regression	
Used in	district_intel/dintel/classify.py
Purpose	Put free text ('Other' complaints, news) into the standard category list.
Simple picture	Instead of reading whole words, it looks at small pieces of letters. 'drainage', 'drainge' and 'draineage' share most pieces, so a spelling mistake still looks like drainage.
How it works	1) Split text into overlapping chunks of 2 to 5 characters inside words. 2) TF-IDF gives each chunk a weight: high if it is frequent in this text but rare overall. 3) Logistic Regression learns how much each chunk points to each category and outputs a probability per category. 4) If the best probability is below 0.45 the record stays 'Other'. Structured sources use lookup tables (crosswalks) instead.
Real settings	TfidfVectorizer(analyzer='char_wb', ngram_range=(2,5), min_df=2, max_features=200,000, sublinear_tf=True); LogisticRegression(C=6.0, class_weight='balanced', max_iter=3000); confidence cut 0.45. Trained on labelled grievance, police and PWD text.
Why we chose it	Complaint text is short, noisy, and mixes Tamil, Tanglish and spelling errors.
Why it is better	Character pieces survive typos that break word models and keyword lists. Much smaller and faster than BERT, runs on a laptop CPU, gives probabilities for the confidence cut. 'Balanced' weights stop big categories swamping small ones.
Result	About 0.99 accuracy on template text. We say honestly that real citizen text will score lower.

A4. News incident filter (weak supervision + 300 hand labels)	
Used in	district_intel/dintel/loaders/news.py
Purpose	Decide if an article reports a real incident a department must act on (flooding, accident, collapse) or not (power-cut schedule, politics, court case).
Simple picture	First, rough rules label thousands of articles quickly (some wrong). Then people carefully label 300, focusing on the doubtful ones. The model learns from both.
How it works	1) Rules create a large, noisy training set (weak labels). 2) 300 hand labels by stratified sampling: 150 accepted, 75 just below the threshold, 75 far below. 3) A classifier is trained on both. 4) Scored with 5-fold cross-validation, weighted by stratum, so it is never tested on its own training labels.
Why we chose it	Most news is not an incident; showing all of it would bury the Collector.
Why it is better	Versus rules only: F1 rose from 0.41 to 0.63. Versus labelling thousands by hand: weak supervision gets most of the value from only 300 labels, and stratified sampling puts labels where the model is unsure.
Result	F1 0.63 (precision 0.69, recall 0.57). It still misses about 4 in 10 incident articles; more labels is the known fix.

A5. Story clustering (character similarity, then multilingual embeddings)	
Used in	district_intel/dintel/loaders/news.py, dintel/embed.py
Purpose	Group articles about the same event ('developing stories') and count distinct outlets.
Simple picture	Ten newspapers write about the same accident in different words. Embeddings turn each headline into a point in 'meaning space'; headlines that mean the same thing land close together.
How it works	1) One document per URL; Google News copies merged into the publisher's copy. 2) Stage 1: cheap character similarity catches near-identical headlines. 3) Stage 2: multilingual-e5-small embeddings; two articles join if cosine similarity is at least 0.95 and they are within 24 hours. 4) Groups are formed with union-find (see A8).
Real settings	Model intfloat/multilingual-e5-small (cached on disk); cosine 0.95; 24-hour window.
Why it is better	Embeddings catch reworded headlines that letter-matching misses; the cheap first stage saves work. e5-small is free, runs offline on a CPU and understands Tamil, unlike paid cloud embeddings. The strict 0.95 prefers missing a merge over a wrong merge.
Result	129 same-language merges, all checked by hand and correct. English-Tamil merging is switched off: headlines alone could not separate the same event from two different accidents on the same day.

Group 2: Location, merging and linking

A6. Geo-location: point-in-polygon, snapping, taluk majority vote	
Used in	district_intel/dintel/geo.py, geolocate.py
Purpose	Put every record in the correct ward, zone and taluk so maps and filters are right.
Simple picture	Each ward is a shape on the map. 'Point-in-polygon' asks: is this dot inside this shape? (Draw a line from the dot; if it crosses the border an odd number of times, it is inside.)
How it works	1) Records with coordinates: test against the 200 GCC ward polygons (matplotlib.path). 2) Points up to 500 m outside a ward are snapped in and flagged (confidence 0.8). 3) Records without coordinates: located from street (about 300 m precision), locality (800 m) or area (1,500 m), and that precision is stored. 4) Ward to taluk by majority vote of police and PWD records that carry a taluk code. Confidence below 0.7 goes to the review queue.
Why it is better	Real ward geography instead of invented regions. matplotlib.path needs no GEOS/GDAL install (a common Windows problem with Shapely/GeoPandas). Snapping rescues GPS points just outside a boundary. Stored precision lets linking widen its search for vague locations.

A7. Scenario overlay: one shared world for synthetic data	
Used in	district_intel/dintel/world.py, dintel/sync.py
Purpose	Make separately generated synthetic sources agree with each other and with real weather.
Simple picture	Four people each wrote a diary of the same 6 months, but each invented different rainy days. We give them one shared weather calendar so their diaries agree.
How it works	1) A world calendar collects rain days from every source (grievances, police, PWD flood spikes, IMD warnings). 2) The same rain days are passed to every generator through its own options. 3) On a world rain day, a source with no flood response gets the reports it would have produced; existing records are adopted, not duplicated. 4) Every planted row is flagged (is_synthetic = 1, is_overlay = 1) and a truth file records which world event it belongs to. Seed 2026, 4 to 9 flood incidents per rain day, 70 other planted incidents.
Why it is better	Before this, flood-day agreement between sources was at most 0.12, so linking could not be tested. Generators stay untouched, real sources are never changed, and the truth file gives ground truth to measure dedup and linking accuracy, which most prototypes cannot do.
Result	Flood agreement per 3-day episode: 0.67 / 0.64 / 0.91.

A8. Duplicate removal: blocking + union-find	
Used in	district_intel/dintel/dedup.py
Purpose	Merge repeated reports from the same source (for example 5 citizens reporting one pothole).
Simple picture	Union-find is like friendship groups: if A is a duplicate of B and B of C, then A, B and C are all one group, even though A and C were never compared.
How it works	1) Blocking: only compare records with the same category, nearby and close in time (not every pair). 2) Source rules decide duplicates (the portal's rule, officers' own 'Duplicate of' decisions, police and PWD rules). 3) Union-find joins all duplicate pairs into groups.
Why it is better	Blocking avoids comparing every pair (n-squared work becomes almost linear). Union-find is transitive and almost instant per merge, and keeps every member traceable, unlike simply deleting duplicates.

A9. Cross-source linking: KD-tree + Logistic Regression + learned threshold	
Used in	district_intel/dintel/linking.py
Purpose	Recognise that a grievance, a police report, a PWD incident and a news story are the same real-world incident.
Simple picture	Like a detective matching reports: same kind of problem, same place, same time, similar words, so probably the same event. The detective gives a confidence score; unsure cases go to a person.
How it works	1) Blocking: only compatible category groups (flood and drainage are both 'water'), within each category's time window (+48 h for news) and distance (+ both points' location precision). A KD-tree (scipy cKDTree) quickly finds nearby points. 2) Features per pair: distance, time gap, TF-IDF text similarity, same category, same ward, coarse location, has-news. 3) Logistic Regression (balanced) trained on half of the world events; the other half is kept for testing. 4) Merge threshold chosen to give the best F1 on the training half. 5) Bands: 0.85 or more = link; 0.40 to 0.85 = review queue (Linker agent, then a person); below 0.40 = no link. 6) Union-find builds incidents. Each pair keeps its features, so the link can be explained.
Real settings	link_merge_threshold 0.85, link_review_threshold 0.40 (config.yaml).
Why we chose it	Record IDs never match across departments (CMP-..., GRV-..., 2026-NNN), so linking must use place, time and text.
Why it is better	KD-tree keeps cost proportional to neighbours, not every pair. Logistic regression gives a calibrated probability and readable weights, so a reviewer can see why two records were linked, unlike a neural network. The grey band sends uncertain cases to people. Held-out testing proves the number.
Result	Precision 0.99, F1 0.88 on held-out events.

A10. Severity and explainable priority score	
Used in	district_intel/dintel/severity.py, incidents.py
Purpose	Rank incidents for the Collector and explain the rank in plain words.
Simple picture	Like a hospital triage card: points for how dangerous, plus points for each warning sign, and each point comes with its reason written down.
How it works	Severity (0 to 100) from fixed facts: category risk (high-risk issue +35, public safety/sanitation +25, civic +15, administrative +5), immediate hazard +15, people injured or dead, vulnerable victim +10. Priority = severity + bonuses, each adding a reason (see formula). Deadlines come from each category's target hours.
Formula	$\text{priority} = \text{severity} + 5 \times \log_2(\text{sources}) + 3 \times \log_2(1 + \text{citizen complaints}) + 10$ if past deadline (20 if over 2x) + 8 if Severe/High and not verified + 8 if 3+ new reports in 24 h + 5 each for: vulnerable people, 2+ news outlets, only in news, rain-linked, 3+ similar incidents in 90 days.
Why it is better	Versus a black-box machine-learning ranking: every point is explainable and auditable (shown as reasons in the pop-up), needs no training data, and weights are tunable in YAML. log2 stops one viral issue with 100 complaints drowning everything else. Priority is recomputed at the current time so it never goes stale.

Group 3: Statistics: spikes, hotspots, trends

A11. Spike detection: Poisson baseline with a weekday factor	
Used in	district_intel/dintel/analytics.py (anomalies)
Purpose	Detect an unusual rise in incidents of a category in a zone on a day.
Simple picture	If a zone normally has about 2 drainage complaints a day, then 3 is normal but 9 is very unlikely by chance. The Poisson model tells us exactly how unlikely.
How it works	1) Daily counts per category x zone. 2) Expected count (lambda) = mean of the previous 28 days (needs at least 14 days), multiplied by a district-wide weekday factor (some weekdays are always busier). 3) p-value = chance of seeing this many or more if nothing unusual is happening. 4) Spike if $p < 0.01$ and the count is at least 3.
Real settings	baseline_days 28; $p < 0.01$; minimum count 3; scipy.stats.poisson.
Why it is better	Versus a fixed threshold ('more than 10'): adapts to each zone and category. Versus mean + 2 standard deviations: correct for small counts, where the normal curve fails. The weekday factor stops normal busy days being flagged; the minimum of 3 stops alarms for 0 to 1. Output shows observed, expected, ratio and p-value, so it is fully explainable.

A12. Hotspots: DBSCAN with haversine distance	
Used in	district_intel/dintel/analytics.py (hotspots)
Purpose	Find places where one kind of incident keeps happening (for example a junction that floods every rain).
Simple picture	Drop all incident points on the map. Wherever at least 5 points sit within 250 m of each other, that is a crowd, a hotspot. Lonely points are ignored as noise.
How it works	Per category, take the last 90 days of incident points. DBSCAN with the haversine metric (real distance on the earth's curved surface), eps = 250 m, min_samples = 5, using a ball-tree index. Each cluster becomes a hotspot with its centre and count.
Why it is better	Versus K-means: no need to choose the number of clusters, finds any shape, and ignores scattered noise. Haversine gives true metres. 250 m is about a street block, matching a field officer's site.

A13. Hot wards: Getis-Ord Gi* statistic	
Used in	district_intel/dintel/analytics.py (gi_star)
Purpose	Find wards that are statistically hot together with their neighbours , not just high on their own.
Simple picture	One busy ward could be chance. A busy ward surrounded by busy wards is a real hot area.
How it works	Incident counts per ward over 30 days. Neighbours = wards that share a border (including the ward itself). The Gi* z-score compares the local total with the district average. Hot when $z > 1.96$ (95% confidence).
Why it is better	Gives statistical significance instead of eyeballing a colour map (big wards naturally have more incidents). It is a standard method in crime and health mapping, and complements DBSCAN (point clusters) with area-level evidence.

A14. EWMA and z-score for sensor series	
Used in	district_intel/dintel/analytics.py (observation_signals)
Purpose	Show whether lake levels, AQI, rainfall or hospital beds are unusually high or low now.
Simple picture	EWMA is a smoothed line that gives more weight to recent days. The z-score says how many 'normal swings' today is away from the usual level.
How it works	For each metric: EWMA with span 7 (smoothed trend) and the z-score of the latest value against the last 28 days' mean and standard deviation. Quality flags mark gaps and suspicious readings.
Why it is better	EWMA reacts faster than a simple moving average while still smoothing noise. The z-score puts different units (metres, AQI, %) on one comparable scale.

Group 4: Agents, the assistant and data delivery

A15. Agents with a numeric verifier (Briefing, Steward, Planner, Watchdog)	
Used in	district_intel/dintel/agents/
Purpose	Write daily/weekly/monthly/quarterly briefings, draft action plans from playbooks, and check data health.
How it works	Agents are rule-based programs that read the computed tables and fill templates. If an AI model is enabled, it may only reword the text, and the result is accepted only if the verifier finds every number in the source data. AI calls are capped per run.
Why it is better	Versus letting an AI write the briefing: no invented numbers, works with no API key, cheap. Every AI step has a deterministic fallback.

A16. Gap finder ('Only in news')	
Used in	district_intel/dintel/agents/workers.py; Collector page 2
Purpose	Show problems reported in the news that no department has recorded.
How it works	A news incident the linker could not join to any department record (media_only) becomes a gap. It gets +5 priority and is routed to the right department's officer using its department tag.
Why it is better	Reuses the linking result, so no extra model; turns news from 'reading' into an action list. This is the blind spot a Collector most wants to know.

A17. Ask District IQ: guarded tool pipeline with a SQL compiler	
Used in	chennai-grievance-portal-main/src/lib/assistant/ (pipeline.ts, guard.ts, compile.ts, tools.ts)
Purpose	Answer free questions safely, with real numbers.
How it works	1) Guard (plain code, before any model) refuses personal data, data changes and prompt injection. 2) Router: a fast model picks a console tool; without a key, rules do. 3) For new groupings the model fills a QueryPlan (it never writes SQL); the compiler turns it into one parameterised SELECT using only catalogued tables and declared joins, a 5-second time limit, row caps, on a read-only connection; errors go back for at most 2 repairs. 4) Facts go to the composer, which writes the answer. 5) Officers: lockDept forces their department on every tool and query, on the server.
Why it is better	Versus text-to-SQL (the model writes SQL): no SQL injection, no table outside the catalogue, no runaway query. Versus a plain chatbot: answers come from data, not model memory. The department lock cannot be bypassed by a prompt.

A18. Number verifier and chart checker	
Used in	src/lib/assistant/verify.ts, chartspec.ts
Purpose	Guarantee that no number in an AI answer is invented.
How it works	Every number in the headline, text and chart titles is extracted and must match a returned fact at the precision written ('97.7%' and '98%' both match 97.65). Allowed extras: dates, times, IDs, zone/ward numbers, window lengths (7, 30, 90 days), numbers from the question. Anything else fails the answer (one retry, then a template answer). The chart spec is validated before display.
Why it is better	Most chatbots trust the model; ours proves each number. This is the key trust feature.
Result	Covered by the 186 Vitest tests and a golden-question evaluation set (eval/golden.json).

A19. Typo and Tanglish handling: Damerau-Levenshtein spell correction	
Used in	src/lib/assistant/fuzzy.ts, spell.ts, lang.ts
Purpose	Understand 'incidnets in adyr' or 'thakkali vilai' (tomato price).
Simple picture	Edit distance counts how many single changes turn one word into another: 'adyr' to 'adyar' is 1 insert. Damerau also counts swapping two letters as 1 change ('teh' to 'the').
How it works	Words are corrected only toward the district's own vocabulary (places, categories, commodities, console terms), only when one candidate is clearly closest (stop after distance 3), and never when the word is a real English word (274,000-word list) or romanised Tamil. Tamil, Tanglish and Hindi commodity names are mapped. The answer shows 'Understood as ...'.
Why it is better	Generic spell-checkers would 'correct' Chennai place names into English words. Refusing to guess between equal candidates avoids wrong corrections; showing the correction keeps it transparent.

A20. Atomic swap, diff-only writes and content-addressed snapshots	
Used in	dintel/mysql_export.py (and, in the AWS version, aws/export_intel.py and the collect Lambdas)
Purpose	Update data stores quickly and cheaply, without readers ever seeing half-loaded data.
How it works	MySQL: load into new '___new' tables, then one atomic RENAME swaps them all at once; row and per-column counts are compared with SQLite (mismatch = fail). AWS: DynamoDB gets only rows that changed; S3 stores each table as gzip JSON named by its SHA hash, unchanged tables are skipped, and the manifest intel/current.json is switched last.
Why it is better	No downtime and no partial reads while people use the dashboard; far fewer writes (lower cost); hashes make uploads safe to retry.
Result	On this PC: 31 tables, about 300,000 rows loaded in under a minute, zero mismatches.

If they ask...

Q: Why logistic regression and not deep learning?

A: Small data, CPU only, and above all explainable: we can show why two records were linked or why a category was chosen. For government decisions, explainability matters more than a few points of accuracy.

Q: Why Poisson for spikes?

A: Daily incident counts are counts of rare events, which is exactly what the Poisson distribution models. For small numbers, mean + 2 SD gives false alarms. We add a weekday factor and a minimum of 3.

Q: Why DBSCAN and not K-means?

A: K-means needs the number of clusters in advance and forces every point into a cluster. DBSCAN finds any shape, ignores noise, and with haversine works in real metres.

Q: How did you measure linking accuracy without real ground truth?

A: The shared world calendar plants matching flood reports across synthetic sources and records which event each belongs to in a truth file. We train on half the events and test on the other half.

Q: How does linking work if IDs don't match?

A: By place, time and text: KD-tree finds nearby candidates, logistic regression scores each pair, a learned threshold decides, uncertain pairs go to human review, union-find builds the incident.

Part 7. Storage: where the data lives

Store	Owner	Contents	Why
SQLite district_intel.db	Pipeline	32 tables + 6 views (about 160 MB)	Zero setup, one file, perfect for a batch build and tests.
MySQL district_intel	Pipeline	31 tables, about 300,000 rows; replaced each export by an atomic swap	Many users can read at once; readers never see half-loaded data.
MySQL district_intel_ops	Dashboard	collector_decisions, action_updates, audit_log, officer_reports, sources, mandi prices, assistant tables	Never dropped or overwritten: it holds people's decisions.
MySQL district_collector_dashboard	Portal	users, complaints, complaint types, areas, localities, streets, wards, zones	The citizen portal's own data.

Say this in the review

We separate what the pipeline owns from what people own. The pipeline can rebuild its store every hour, but it can never erase a Collector's decision, because decisions live in a different database.

Two versions: MySQL (this PC) and AWS

The challenge asked for the platform to run in FarmwiseAI's AWS account using only S3, Lambda, API Gateway, DynamoDB, CloudWatch Logs and Bedrock (region ap-south-1). So the team also built an AWS version (kept in a separate repository, farmwise-aws; the aws/ folder was removed from this repository in commit 548c962). In that version:

- Collectors run as **Lambda** functions (one package deployed 5 times with a SOURCE setting: imd, cpcb, cfm, pwd, hospital, plus news).
- **API Gateway** routes (POST /refresh/<source> and others) need a secret x-refresh-key header and are throttled to 1 request per second.
- **GitHub Actions** is the timer, because EventBridge was not allowed for the team's AWS role.
- Data in **S3** (content-addressed snapshots) and **DynamoDB** (diff-only writes; 27 tables, about 58,000 rows migrated).
- The website loads the S3 snapshot into an **in-memory SQLite** database, and a dialect layer rewrites the MySQL queries (src/lib/aws/dialect.ts), switched by DATA_BACKEND=aws. The same queries work in both versions.
- The heavy ML build (sentence-transformers) is too big for Lambda, so the **PC builds and AWS stores**.
- AI runs on **Amazon Bedrock** (Nova Lite, Nova Micro, Ministral), so data stays inside AWS.

If they ask...

Q: Why two versions?

A: MySQL is the simplest way to run locally. The challenge required AWS with only certain services (no RDS), so the AWS version uses S3 + DynamoDB + in-memory SQLite, and the same queries run through a dialect layer.

Q: What does it cost to run?

A: Everything in the cloud version is serverless and pay-per-use: Lambda per run, DynamoDB per request, a few tens of MB in S3, throttled API Gateway. No always-on servers.

Part 8. Security and privacy

Risk	What we did
Stolen passwords	Passwords hashed with bcrypt; never stored in plain text.
Session theft	JWT in httpOnly cookies (page scripts cannot read them); separate cookie per portal (dda6e99).
Wrong user seeing a page	Middleware checks the role before every page; /citizen, /officer, /collector protected.
Officer seeing another department	Department read from the database on every request, never from the URL; other departments' records return 404.
SQL injection	Parameterised SQL only; the assistant's SQL is compiled from a plan, catalogue-only, read-only, 5-second limit, row caps.
Personal data	Mobile numbers replaced by keyed hashes in the intelligence store; Aadhaar encrypted at rest; the assistant refuses personal-data requests.
Brute force	Login limits: 20 per IP and 8 per email per 15 minutes; assistant limits per user.
Prompt injection	Guard runs as code before the AI; model never writes SQL; department lock in server code.
Secrets	.env and officer passwords are git-ignored; AWS keys never reach the browser.
Unsafe scraping	Collectors never bypass CAPTCHAs or logins; HTTPS always verified (truststore).

Part 9. Tech stack: what, how we used it, why, why better

9.1 Data collection

Technology	How we used it	Why	Why better than the alternative
Python 3.11/3.12	All collectors, news pipeline, district_intel	Best data and ML ecosystem	vs Java/Node: far richer data, geo and ML libraries; less code
requests + feedparser	HTTP calls to IMD/CFM/AGMARKNET; reading RSS	Simple, reliable, key-free	vs paid news APIs: free, no quota, includes Tamil outlets
trafilatura	Pulls the article body out of news pages	Best open-source article extractor	vs hand-written BeautifulSoup: removes menus and ads for any site
Playwright	Reads CPCB's JavaScript map	The page has no open API	vs Selenium: auto-waits, faster, fewer flaky runs
langdetect, tzdata, dateutil	Tamil/English detection; all times in Asia/Kolkata	Correct language rules and IST	vs manual parsing: handles many date formats
truststore	Windows certificate store for HTTPS	Antivirus inspects TLS on the PC	vs verify=False: stays secure
Node.js + TypeScript	Police dataset generator	Same language as the portal	vs plain JS: type errors caught early

9.2 Intelligence layer

Technology	How we used it	Why	Why better than the alternative
pandas / NumPy	Load 8 sources into one 70-column schema; joins, group-bys, rolling windows	Fast table processing	vs loops/Excel: much faster and repeatable
scikit-learn	TF-IDF, Logistic Regression (classifier, linker), DBSCAN, cross-validation	Proven, explainable ML	vs deep learning: works on small data, CPU-only, readable weights
SciPy	Poisson p-values, normal cut-offs, cKDTree neighbour search	Correct statistics, fast search	vs hand-written maths: tested and accurate
sentence-transformers (multilingual-e5-small)	Embeddings for English and Tamil headlines	Multilingual, small, offline	vs OpenAI embeddings: free, private, no internet at build time
matplotlib.path	Point-in-polygon for 200 ward shapes	No native GIS install	vs Shapely/GeoPandas: no GEOS/GDAL problems on Windows
PyYAML	All thresholds, categories, deadlines, playbooks in config.yaml / reference/*.yaml	Tune without code changes	vs hard-coded values: reviewers can read and change rules
SQLite	Pipeline output store	Zero-config single file	vs a DB server for batch output: portable
pytest	24 district_intel + 88 news pipeline tests	Catch regressions	vs manual checks: runs in seconds

9.3 Web application

Technology	How we used it	Why	Why better than the alternative
Next.js 14 (App Router) + React 18	All pages and REST APIs in one app; middleware for roles	Full-stack in one project	vs React + separate Express: one deployment, shared types, server rendering
TypeScript	All portal code	Type safety on a large codebase	vs JavaScript: errors caught before running
Tailwind CSS + scoped CSS	Portal styling; fixed-screen console that scales	Fast, consistent UI	vs Bootstrap: lighter, fully custom
MySQL 8 + mysql2	Store, parameterised SQL, no ORM	Concurrent reads and writes	vs ORM: transparent, fast SQL; no injection
Leaflet + OpenStreetMap / Esri imagery	Complaint pin map; Collector satellite map with ward outlines	Free maps, no key	vs Google Maps: no billing account or API key
ECharts	Charts in assistant answers	Many chart types, fast	vs Chart.js: richer types, more data
jsPDF + autotable	Briefing PDF in the browser	No server load	vs server PDF: instant, works offline
bcryptjs, jose (JWT), zod	Password hashing, signed cookies, input validation	Standard security	vs tokens in localStorage: safe from XSS
nodemailer	OTP and password-reset emails	Simple SMTP	vs paid email services: no lock-in
tesseract.js	OCR of scanned newspaper pages (English + Tamil) in the browser	Free, in-browser	vs cloud OCR: no cost, not sent anywhere
Vitest	186 tests for compiler, verifier, chart checker, guards, typos	Fast TypeScript testing	vs Jest: faster, native TypeScript/ESM

9.4 AI layer

Technology	How we used it	Why	Why better than the alternative
Vercel AI SDK (ai, @ai-sdk/*)	Streaming answers; switch Bedrock / Groq / OpenAI by config	One API for many models	vs separate SDKs: change provider without code changes
Amazon Bedrock (Nova Micro, Nova Lite, Ministral 3B/8B)	Router (fast, cheap model) and composer	Allowed models; data stays in AWS	vs OpenAI: in-region, billed to the team account
@huggingface/transformers (multilingual-e5-base)	Local multilingual embeddings for meaning search	Local and free	vs hosted embeddings: no cost, private
word-list	274,000 English words so the spell-checker does not 'correct' real words	Avoid wrong corrections	vs no list: fewer false corrections

9.5 Operations

Technology	How we used it	Why
Windows Task Scheduler	Runs run_intel.bat hourly and at sign-in	Free; no always-on server needed
Git + GitHub (+ Git LFS)	Version control; large files (model, news, database) in LFS	Review, history, sharing large files
AWS Lambda, API Gateway, S3, DynamoDB, CloudWatch, GitHub Actions	Cloud version (separate repository)	Serverless, pay-per-use, allowed by the challenge

Part 10. Key design decisions (why we built it this way)

Decision	Alternative we rejected	Why our choice is better
Numbers from SQL and statistics; AI only for language	Let the AI compute and write numbers	No invented figures; a verifier proves every number
Explainable models (logistic regression, rule scores)	Deep neural networks	Small data, CPU-only, and reviewers can see why
Cross-source linking with a human review band	Fully automatic merging	Uncertain cases go to people; precision 0.99
Collectors never modified; fixes in a separate layer	Edit each collector for the dashboard	Sources stay testable; one place to fix data
Pipeline store separate from the dashboard ops store	One shared database	Rebuilds never erase people's decisions
Real ward geography (200 GCC wards)	Invented regions	Maps and filters match how Chennai is governed
Measured accuracy with held-out ground truth	No evaluation	We can state numbers (F1 0.88, 0.63) and limits honestly
Fixed one-screen console, filters instead of per-department pages	Long scrolling dashboard, many pages	The Collector sees everything at once
Rolling 24-hour Daily window (dda6e99)	Today from midnight	Always a full day of data, any time it is opened
Hourly collection (dda6e99)	Once a day at 6 AM	Fresher data; a PC that was off catches up
Serverless AWS with an external timer	Servers or EventBridge	Fits the allowed services and budget

Part 11. Testing, results and honest limits

How we know it works

Measure	Result
Automated tests	About 300: 24 district_intel (pytest), 88 news pipeline (pytest), 186 website + assistant (Vitest), plus a golden-question evaluation set for the AI
Cross-source linking	Precision 0.99, F1 0.88 on held-out events
News incident filter	F1 0.63 (precision 0.69, recall 0.57), up from 0.41 with rules only
Story clustering	129 merges checked by hand, all correct
Category classifier	About 0.99 on template text (lower expected on real text)
Scenario overlay	Flood agreement 0.67 / 0.64 / 0.91 per 3-day episode (was at most 0.12)
MySQL export	Row and per-column counts compared with SQLite on every export; zero mismatches on this PC

Honest limits (say these before they ask)

- Department data (police, PWD, hospitals, most grievances) is synthetic until real systems are connected.
- The news filter still misses about 4 in 10 incident articles; more hand labels is the fix.
- English-Tamil story merging is switched off because headlines alone were not reliable enough.
- About 98% of news items are headline snippets, not full text.
- Rainfall and AQI history only grows from our own runs (no open history endpoints); some station coordinates are approximate.
- On this PC: the grievance generator step of the refresh failed (the rest of the data refreshed fine), and no AI keys are set, so the assistant uses rule mode.

What we would improve next

- Connect real department data.
- More hand labels for the news filter.
- Full-text English-Tamil merging.
- Move the ML build into the cloud.
- More districts: wards, taluks, departments and playbooks are reference files, not code.

Part 12. Likely reviewer questions and answers

Q. What problem does District IQ solve?

The Collector gets information from many disconnected departments and the news; the same incident appears many times in different formats. We unify it into one trustworthy list of real incidents with priorities and reasons, route the work, and track it to verified completion.

Q. What is the single most important design idea?

Numbers come from SQL and statistics, never from an AI model. The AI only understands questions and writes sentences, and a verifier checks every number.

Q. Why not just use ChatGPT for everything?

AI models can produce confident wrong numbers. We use AI only for language, and the whole system still works without any AI key.

Q. How do you collect data?

One independent program per source: RSS for news, JSON APIs for IMD, the flood monitor and AGMARKNET, Playwright for CPCB's map, and generators for police, PWD, hospital and grievances. Hourly, with 3 retries.

Q. How do you know a news article is about Chennai?

A gazetteer of Chennai places in English and Tamil, plus exclusion rules for other districts (A1).

Q. How do you classify complaints with spelling mistakes or Tanglish?

Character n-gram TF-IDF with logistic regression (A3): it looks at letter pieces, so typos still match.

Q. How do you put records on the map?

Point-in-polygon against the 200 real GCC ward boundaries, snapping points up to 500 m outside, and storing precision for text-only locations (A6).

Q. How do you remove duplicates?

Blocking (only compare nearby, same-category, same-time records) and union-find to group them (A8).

Q. How do you know a police report and a grievance are the same incident?

KD-tree finds nearby candidates; logistic regression scores place, time, text and category; 0.85+ links, 0.40 to 0.85 goes to human review (A9). Precision 0.99.

Q. How is priority decided?

Severity from fixed facts plus additive bonuses (deadline passed, many sources, news coverage, rain-linked...), each with a written reason (A10). Fully explainable.

Q. How do you detect spikes?

Poisson model: expected count from the last 28 days with a weekday factor; spike if $p < 0.01$ and at least 3 cases (A11).

Q. How do you find hotspots?

DBSCAN with haversine distance, 250 m and at least 5 points over 90 days (A12), plus Getis-Ord G_i^* for hot wards (A13).

Q. What does 'Only in news' mean?

News incidents that no department has recorded: the Collector's blind spot. Found by the Gap Finder and routed to the right department (A16).

Q. What does Daily mean on the dashboard?

Since the latest commit: the last 24 hours ending now. Weekly, Monthly, Quarterly are the last 7, 30 and 90 days.

Q. Why might two computers show different numbers?

The windows count back from the current time, and each PC's data depends on when its pipeline last ran. With the same data and the same time window, the numbers match.

Q. How is the officer prevented from seeing other departments?

The department is read from the database on every request, never from the URL, and the assistant's department lock is in server code.

Q. How does the closed loop work?

Officer approves, works, sends a completion report with a site photo; it appears first in the Collector's My Tasks; the Collector verifies or sends back; every step is in the audit log.

Q. How do you stop the AI being tricked?

The guard runs as code before the model; the model never writes SQL so it cannot reach tables outside the catalogue; the department lock is enforced on the server.

Q. How does the assistant understand typos and Tanglish?

Damerau-Levenshtein distance toward Chennai's own vocabulary only, never 'correcting' real English words, and it shows 'Understood as ...' (A19).

Q. Why MySQL and SQLite both?

SQLite is a zero-setup file for the batch build and tests; MySQL serves many users at once. Export uses an atomic swap so readers never see half-loaded data.

Q. Why Next.js?

Pages and APIs in one TypeScript project with middleware for roles: one deployment, shared types, server rendering.

Q. Why Leaflet and not Google Maps?

Free, no API key or billing account, and works with our own ward polygons and satellite imagery.

Q. Can it work for another district?

Yes. Wards, taluks, departments, categories and playbooks are reference files; a new district needs its reference data and connectors.

Q. What are the limits?

Synthetic department data, news filter misses about 4 in 10, English-Tamil merging off, history only from our runs. Each is on the roadmap.

Q. How many tests?

About 300 automated tests plus a golden-question evaluation set for the assistant.

Glossary: technical words in plain English

Word	Meaning
Pipeline	A chain of steps that data goes through, each step doing one job.
Collector (program)	A program that fetches data from one source and saves it as files.
Gazetteer	A list of place names used to recognise locations in text.
TF-IDF	A way to weight words or letter pieces: important if frequent in this text but rare in all texts.
n-gram	A chunk of n consecutive characters (or words). 'drain' contains the 3-grams 'dra', 'rai', 'ain'.
Logistic Regression	A simple, explainable model that turns weighted features into a probability between 0 and 1.
Precision	Of the things we flagged, how many were right.
Recall	Of the things we should have flagged, how many we caught.
F1 score	A single score that balances precision and recall (their harmonic mean). 1.0 is perfect.
Cross-validation	Testing a model on data it was not trained on, by rotating which part is held out.
Weak supervision	Training with many rough, rule-made labels instead of only expensive hand labels.
Embedding	A list of numbers representing the meaning of a text, so similar meanings are close together.
Cosine similarity	How closely two embeddings point in the same direction; 1.0 means the same meaning.
Union-find	A fast structure for grouping items: if A=B and B=C, then A, B and C end up in one group.
Blocking	Only comparing records that could plausibly match, to avoid comparing every pair.
KD-tree	A data structure that finds nearby points quickly without checking every point.
Point-in-polygon	Testing whether a map point lies inside a shape such as a ward boundary.
Haversine	The formula for real distance between two latitude/longitude points on the earth.
Poisson distribution	The standard model for counts of random events per day; tells how likely a count is.
p-value	How likely a result is if nothing unusual is happening. Below 0.01 means very unlikely (1 in 100).
DBSCAN	A clustering method that finds dense groups of points and ignores scattered ones.
Getis-Ord Gi*	A spatial statistic that finds areas that are high together with their neighbours.
EWMA	Exponentially weighted moving average: a smoothed line that weights recent values more.
z-score	How many standard deviations a value is from the average; above about 2 is unusual.
Edit distance	Number of single-letter changes needed to turn one word into another.
LLM	Large language model, an AI that understands and writes text (e.g. Bedrock Nova).
Hallucination	When an AI states something false with confidence, such as a made-up number.
Prompt injection	Text that tries to trick an AI into ignoring its rules.
JWT	JSON Web Token: a signed login token the server can verify.
httpOnly cookie	A cookie that page scripts cannot read, protecting the login token.
bcrypt	A slow, salted password-hashing method that makes stolen hashes hard to crack.
Middleware	Code that runs before every page request, used here to check roles.
Parameterised SQL	SQL where user input is passed separately from the query text, preventing SQL injection.
Atomic swap	Replacing all tables in one instant step, so nobody sees half-updated data.
Git LFS	Git Large File Storage: keeps big files outside normal Git, with small pointer files in their place.
Serverless	Cloud code that runs only when called and is paid per use, with no server to manage.

One-page cheat sheet: numbers to remember

Number	Meaning	Number	Meaning
8	source feeds	0.99 / 0.88	linking precision / F1
200	GCC wards (real polygons)	0.63 (from 0.41)	news incident filter F1
15 / 17	zones / taluks	300	hand-labelled news articles
25	department consoles	129	story merges checked, all correct
32 + 6	tables + views in the store	0.95, 24 h	story embedding threshold
about 300,000	rows loaded into MySQL	0.85 / 0.40	link / review thresholds
about 3.5 min	full pipeline build	250 m, min 5, 90 days	DBSCAN hotspot settings
20,600+	news articles	28 days, $p < 0.01$, 3+	Poisson spike rule
about 12,300	police records	$z > 1.96$	Gi* hot ward (95%)
about 8,100	grievances	span 7, 28 days	EWMA / z-score windows
2 to 5, C = 6, 0.45	classifier n-grams, regularisation, confidence cut	500 m	ward snapping distance
about 300	automated tests (24 + 88 + 186)	20 / 300	assistant questions per minute / day
Last 24 h	Daily window (since dda6e99)	Hourly	data collection (since dda6e99)

The story in five lines

Say this in the review

1. Eight disconnected sources are collected automatically, every hour.
2. A Python pipeline cleans, locates, de-duplicates and links them into real incidents.
3. Each incident gets a priority with written reasons; statistics find spikes and hotspots.
4. The Collector sees it all on one screen; officers act and send proof; the Collector verifies; everything is audited.
5. An AI assistant answers questions, but every number it says is checked against the data.